

Plan: Email action flows in NestJS

Objetivo: alinear los correos, los tokens temporales y las acciones del dashboard con la arquitectura actual de NestJS + Astro sin duplicar pantallas ni relajar seguridad. La regla base es: solo confirmación de email y reset de contraseña usan token público de un solo uso; el resto de acciones desde correo abren contexto y luego ejecutan endpoints protegidos por JWT, rol y ownership.

Phases

1) Cerrar el contrato de autenticación y tokens

1. Mantener `POST /auth/confirm-email` y `POST /auth/password/reset` como flujos públicos. Reutilizan los tokens ya existentes en [email-verification-token.entity.ts](#) y [password-reset-token.entity.ts](#).
2. No introducir un token público genérico para confirmación de sesión, rechazo, modificación o evaluación. Esos flujos deben seguir dependiendo de JWT y de validaciones de negocio en backend.
3. Confirmar si Astro realmente reenvía el JWT como Bearer. Hoy el backend valida con [jwt.strategy.ts](#) usando Authorization header, no cookie. Si Astro solo lee `access_token` pero no lo reenvía, hay que resolver ese transporte antes de cualquier cambio de links.

2) Corregir la modificación de sesión

4. Ajustar el contrato de [sessions.controller.ts](#) y [sessions.controller.ts](#) para que el `requestId` que aparece en la URL realmente se use al resolver la request pendiente.
5. Corregir [sessions.controller.ts](#) porque hoy declara una ruta sin `:requestId` pero lee `requestId` del param. Debe quedar consistente con la ruta real o eliminarse el endpoint si no aporta valor.
6. En [session.service.ts](#), reforzar la validación para que la respuesta a una modificación no dependa solo de "hay una pendiente en esta sesión", sino de la request específica y de su expiración.

3) Definir el flujo desde correo hacia frontend Astro

7. Cambiar los links de correo para que lleven a rutas del dashboard con contexto, no con autoridad. Ejemplos: `sessionId`, `requestId`, `action`, `view`, `tab`.
8. El frontend debe usar ese contexto para abrir pantallas o diálogos existentes y, después, cargar datos reales con endpoints autenticados. No debe existir una pantalla duplicada para cada correo si ya hay un componente reusable.
9. Para rutas públicas de auth, mantener los links con token opaco solo en el frontend: `/confirm-email?token=...` y `/reset-password?token=...`.

4) Consolidar notificaciones sin sobrediseñarlas

10. Dejar `settleAll`, el renderizado HBS y la persistencia de notificaciones como están, porque el patrón email + app notification está bien resuelto.
11. Extraer solo la construcción de URLs/contextos si hace falta, para evitar que [notifications.service.ts](#) siga creciendo como módulo de navegación.
12. No mover lógica de autorización ni validación de sesión al servicio de notificaciones; ese servicio debe seguir siendo de entrega, no de decisión de negocio.

5) Seguridad y validación transversal

13. Mantener expiración corta y consumo único en los tokens públicos actuales. Confirmar que el backend invalida tokens anteriores al generar uno nuevo.
14. Revalidar ownership, estado de sesión, estado de request y ventanas temporales en cada endpoint protegido: confirmación, rechazo, modificación, evaluación y cambio de datos.
15. Si Astro usa cookie `access_token` como storage local, añadir o confirmar una estrategia de CSRF si el backend pasa a aceptar cookies para autenticación. Si no, mantener Bearer en requests y no asumir que la cookie por sí sola autentica.
16. Tras reset de contraseña, revocar sesiones activas y auditar la acción como ya hace [auth.service.ts](#).

Relevant files

- [src/modules/auth/entities/email-verification-token.entity.ts](#) y [src/modules/auth/services/email-verification.service.ts](#) — patrón vigente de token público de un solo uso para confirmación de email.
- [src/modules/auth/entities/password-reset-token.entity.ts](#) y [src/modules/auth/services/password-reset.service.ts](#) — patrón vigente de token público de un solo uso para reset de contraseña.
- [src/modules/auth/controllers/auth.controller.ts](#) y [src/modules/auth/services/auth.service.ts](#) — endpoints y lógica que no conviene romper.
- [src/modules/scheduling/controllers/sessions.controller.ts](#) y [src/modules/scheduling/services/session.service.ts](#) — aquí se corrige el uso de `requestId` y se preserva la seguridad de confirm/reject/modification.
- [src/modules/session-execution/controllers/session-execution.controller.ts](#) y [src/modules/session-execution/services/evaluation.service.ts](#) — evaluación debe seguir como flujo autenticado.
- [src/modules/notifications/services/notifications.service.ts](#) — solo cambiar construcción de links/contexto, no el modelo de envío.
- [src/modules/auth/strategies/jwt.strategy.ts](#) — confirma que el backend espera Bearer en header.

Verification

verification

1. Revisar que cada acción tenga un único punto de autoridad: auth público solo para confirm/reset; resto por JWT + rol + ownership.
2. Validar que los links de correo abran el estado correcto en Astro sin necesidad de duplicar vistas ni de ejecutar acciones sin autenticación.
3. Confirmar que el backend no acepte realizar confirm/reject/modification/evaluation únicamente a partir de un token de correo.
4. Verificar que la ruta de modificación use el `requestId` real y no una request pendiente genérica.
5. Comprobar que el transporte del JWT entre Astro y NestJS esté definido antes de tocar enlaces o guards.

Decisions

- Reutilizar endpoints actuales siempre que ya existan y estén protegidos correctamente.
- El mejor encaje para este proyecto es: resolver contexto del link y luego ejecutar endpoints normales autenticados.
- Solo si aparece un caso nuevo que deba ejecutarse sin login, tendría sentido introducir una tabla genérica de tokens de acción: hoy no hace falta